

Adatbázisrendszerek 2 BSc

Féléves Beadandó 2

2026. 05. 05.

PL/SQL

Szállodai nyilvántartó rendszer

Készítette: Kádár Balázs

Mérnökinformatikus Bsc: CFW04U

Miskolc, 2026

Tartalom

1. A témakör ismertetése, megtervezése, leírása.....	3
2. A táblák sémaszerkezete	3
3. A PL/SQL program megtervezése és kódja	6
3.1. Triggerek megtervezése	6
PL/SQL Csomag (Package) tervezése	7
A PL/SQL program extra funkcióinak rövid bemutatása	10

1. A témakör ismertetése, megtervezése, leírása

A féléves feladat célja egy szállodai nyilvántartó rendszer szerver oldali adatbázisának és üzleti logikájának megvalósítása Oracle PL/SQL környezetben. A rendszer célja, hogy nyilvántartsa a szálloda ügyfeleit, az ügyfelekhez tartozó foglalásokat, valamint az ügyfelek által gyűjtött hűségpontokat. A kliens oldali alkalmazások tehermentesítése érdekében a kulcsfontosságú ellenőrzések, az azonosítók generálása és a naplózás közvetlenül az adatbázisban, triggerek és tárolt eljárások formájában került kialakításra.

A tervezés során figyelembe vettem a relációs adatbázisok alapelveit, így az ügyfelek és a foglalások külön táblában kaptak helyet, melyeket idegen kulcs (Foreign Key) köt össze, biztosítva az adatintegritást.

2. A táblák sémaszerkezete

A feladat megvalósításához három táblát és három szekvenciát hoztam létre.

- **ügyfelek:** Ebben a táblában tároljuk a vendégek alapadatait (ugyfel_id, nev, telefonszam, husegpontok, regisztracio_datuma).
- **foglalasok:** A foglalások adatait tartalmazza (foglalas_id, ugyfel_id, szobaszam, erkezes_datuma, ar).
- **ugyfel_naplo:** A biztonsági és adminisztratív követelmények miatt létrehozott naplótábla, amely rögzíti az ügyféladatokon végzett DML műveleteket (log_id, muvelet_tipusa, érintett_ugyfel_id, muvelet_datuma).

A táblákhoz és az elsődleges kulcsokhoz tartozó SQL kód (DDL):

```
CREATE TABLE ugyfelek (  
    ugyfel_id NUMBER PRIMARY KEY,  
    nev VARCHAR2(100) NOT NULL,  
    telefonszam VARCHAR2(20),  
    husegpontok NUMBER DEFAULT 0,  
    regisztracio_datuma DATE  
);
```

```
CREATE TABLE foglalasok (  
    foglalas_id NUMBER PRIMARY KEY,  
    ugyfel_id NUMBER,  
    szobaszam NUMBER,  
    erkezes_datuma DATE,  
    ar NUMBER,
```

```

        CONSTRAINT fk_ugyfel FOREIGN KEY (ugyfel_id) REFERENCES
        ugyfelek(ugyfel_id) ON DELETE CASCADE

    );

```

```

CREATE TABLE ugyfel_naplo (

    log_id NUMBER PRIMARY KEY,

    muvelet_tipusa VARCHAR2(20),

    erintett_ugyfel_id NUMBER,

    muvelet_datuma TIMESTAMP

);

```

Number ↑	Elapsed	Statement	Feedback	Rows
1	0.06	CREATE TABLE ugyfelek (ugyfel_id NUMBER PRIMARY KEY,	Table created.	0
2	0.05	CREATE TABLE foglalasok (foglalas_id NUMBER PRIMARY KE	Table created.	0
3	0.03	CREATE TABLE ugyfel_naplo (log_id NUMBER PRIMARY KEY,	Table created.	0
4	0.01	CREATE SEQUENCE seq_ugyfel_id START WITH 1 INCREMENT BY 1	Sequence created.	0
5	0.01	CREATE SEQUENCE seq_foglalas_id START WITH 1 INCREMENT BY 1	Sequence created.	0
6	0.01	CREATE SEQUENCE seq_log_id START WITH 1 INCREMENT BY 1	Sequence created.	0
7	0.05	CREATE OR REPLACE TRIGGER trg_ugyfel_auto_id BEFORE INSERT	Trigger created.	0
8	0.02	CREATE OR REPLACE TRIGGER trg_ugyfel_kontroll BEFORE UPDATE	Trigger created.	0
9	0.02	CREATE OR REPLACE TRIGGER trg_ugyfel_naplozas AFTER INSERT	Trigger created.	0
10	0.03	CREATE OR REPLACE FUNCTION get_ugyfel_info(p_ugyfel_id IN NU	Function created.	0
11	0.03	CREATE OR REPLACE FUNCTION get_ugyfel_osszkoletes(p_ugyfel_id	Function created.	0
12	0.02	CREATE OR REPLACE PACKAGE szalloda_pkg IS PROCEDURE ugy	Package created.	0
13	0.03	CREATE OR REPLACE PACKAGE BODY szalloda_pkg IS -- 1.	Package Body created.	0

1. ábra: A táblák és szekvenciák sikeres létrehozása a szerveren

3. A PL/SQL program megtervezése és kódja

A rendszer logikáját triggerek, tárolt függvények és egy összefogó csomag biztosítja.

3.1. Triggerek megtervezése

A rendszerben három trigger működik:

1. **trg_ugyfel_auto_id**: BEFORE INSERT trigger, amely a seq_ugyfel_id szekvencia segítségével automatikusan kitölti az elsődleges kulcsot, ha az nem lett megadva.
2. **trg_ugyfel_kontroll**: BEFORE UPDATE trigger, amely ellenőrzi a módosításokat. Megakadályozza, hogy egy ügyfél hűségpontja negatív tartományba lépjen. Ha ez megtörténne, a trigger egy RAISE_APPLICATION_ERROR segítségével megszakítja a tranzakciót.
3. **trg_ugyfel_naplozas**: AFTER INSERT OR UPDATE OR DELETE trigger, amely minden az ügyfelek táblát érintő eseményt lement az ugyfel_naplo táblába, rögzítve a művelet típusát és pontos idejét.

A módosításokat kontrolláló trigger kódja:

```
CREATE OR REPLACE TRIGGER trg_ugyfel_kontroll
BEFORE UPDATE ON ugyfelek
FOR EACH ROW
BEGIN
    IF :NEW.husegpontok < 0 THEN

        RAISE_APPLICATION_ERROR(-20001, 'Hiba: A hűségpont nem lehet negatív!');

    END IF;
```

END;

Tárolt függvények és kurzorok

Két függvényt készítettem a lekérdezések egyszerűsítésére:

- `get_ugyfel_info`: Egy **implicit kurzort** (`SELECT ... INTO ...`) használ, amely visszaadja az adott azonosítójú ügyfél nevét és telefonszámát formázva.
- `get_ugyfel_osszkoltes`: Ez a függvény az **aggregált értékek** számításáért felel, a `SUM(ar)` segítségével kiszámolja, hogy egy adott ügyfél összesen mennyi bevételt generált a szállodának a foglalásai alapján.

Az implicit kurzort használó függvény kódja:

```
CREATE OR REPLACE FUNCTION get_ugyfel_info(p_ugyfel_id IN NUMBER)
RETURN VARCHAR2 IS

    v_nev  ugyfelek.nev%TYPE;

    v_tel  ugyfelek.telefonszam%TYPE;

BEGIN

    SELECT nev, telefonszam INTO v_nev, v_tel FROM ugyfelek WHERE
    ugyfel_id = p_ugyfel_id;

    RETURN 'Név: ' || v_nev || ', Tel: ' || v_tel;

EXCEPTION

    WHEN NO_DATA_FOUND THEN RETURN 'Nincs ilyen ügyfél.';

END;
```

PL/SQL Csomag (Package) tervezése

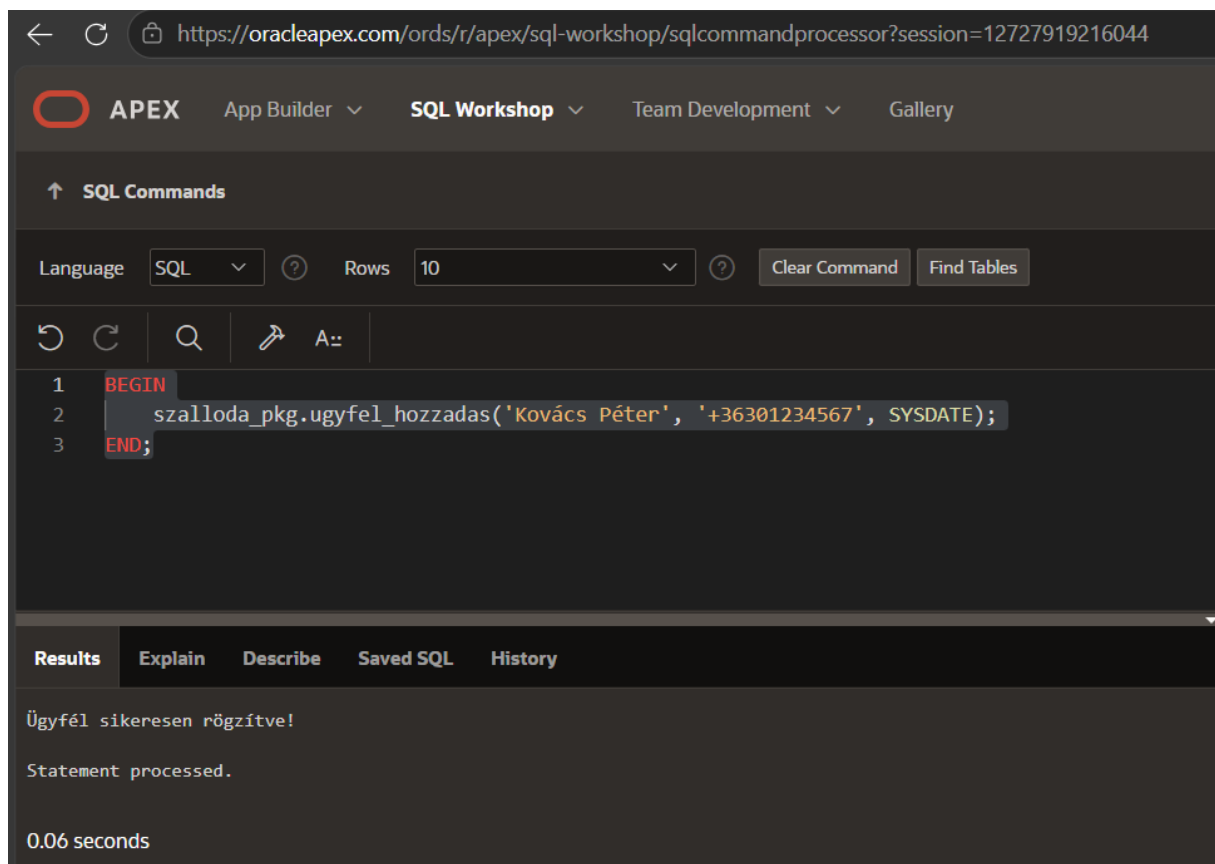
A táblához tartozó CRUD (Create, Update, Delete) funkciókat és az összetett listázást a `szalloda_pkg` nevű csomag fogja össze. A csomag tartalmazza a kivételkezelést is (`EXCEPTION WHEN OTHERS THEN ROLLBACK;`), így hiba esetén nem sérül az adatbázis

konzisztenciája. A csomag része továbbá a foglalások_listazasa eljárás, amely egy **explicit kurzort** használ (OPEN, FETCH, CLOSE), hogy végigiteráljon egy adott ügyfél összes foglalásán, és azt formázva kiírja a standard kimenetre.

A PL/SQL program funkcióinak bemutatása

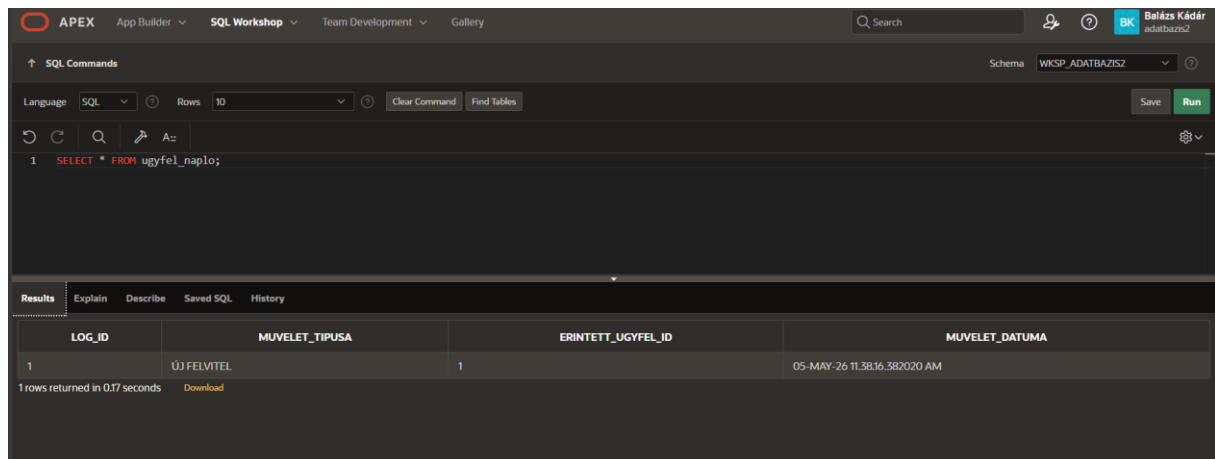
A megírt programkód tesztelése Oracle APEX környezetben történt. Az alábbiakban a főbb funkciók működése látható.

Az adatbázis csomagon keresztül történő adatrögzítése sikeresen lefut



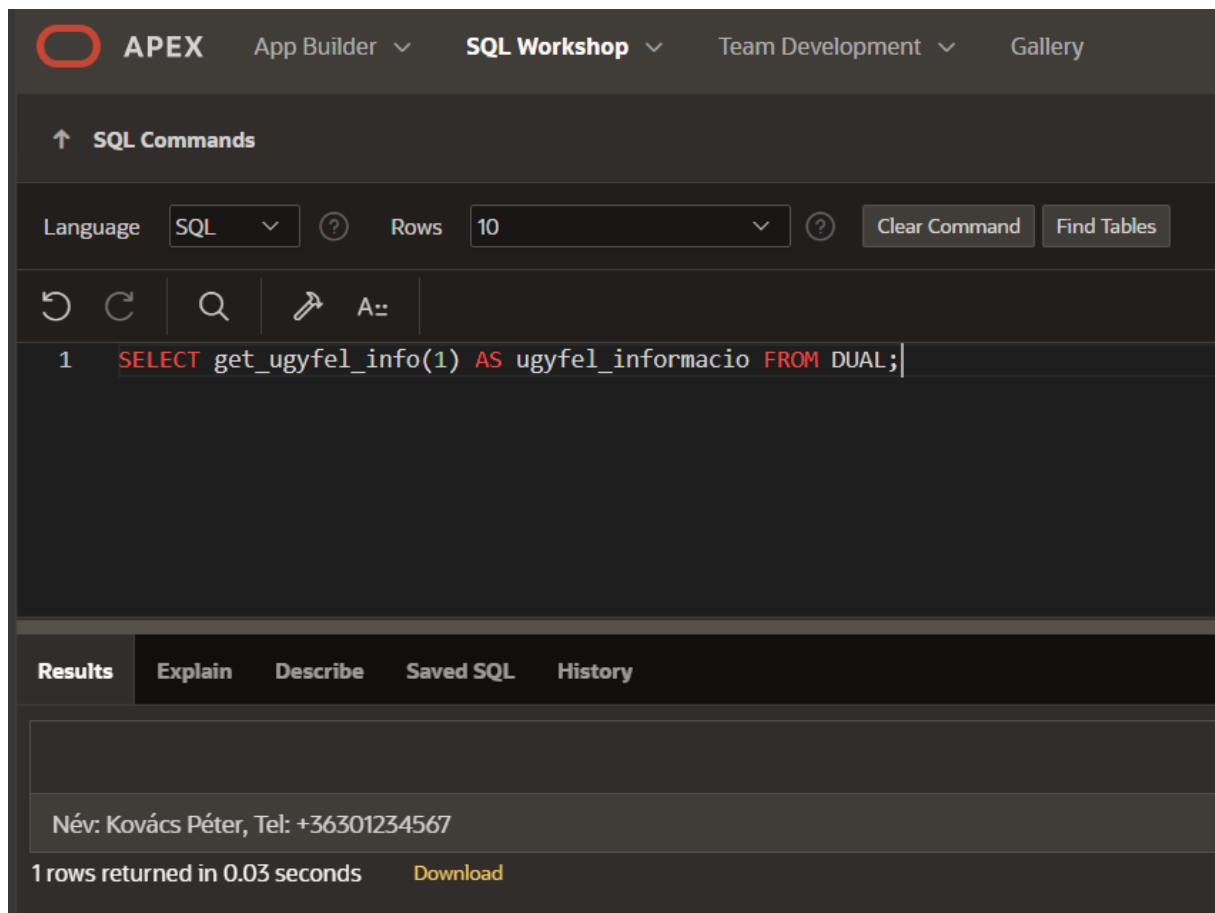
2. ábra: Ügyfél adatainak felvitele a tárolt eljárás segítségével

A naplózó trigger megfelelően működik, a felvételi eseményt automatikusan rögzítette a napló táblában (Lásd: 3. ábra).



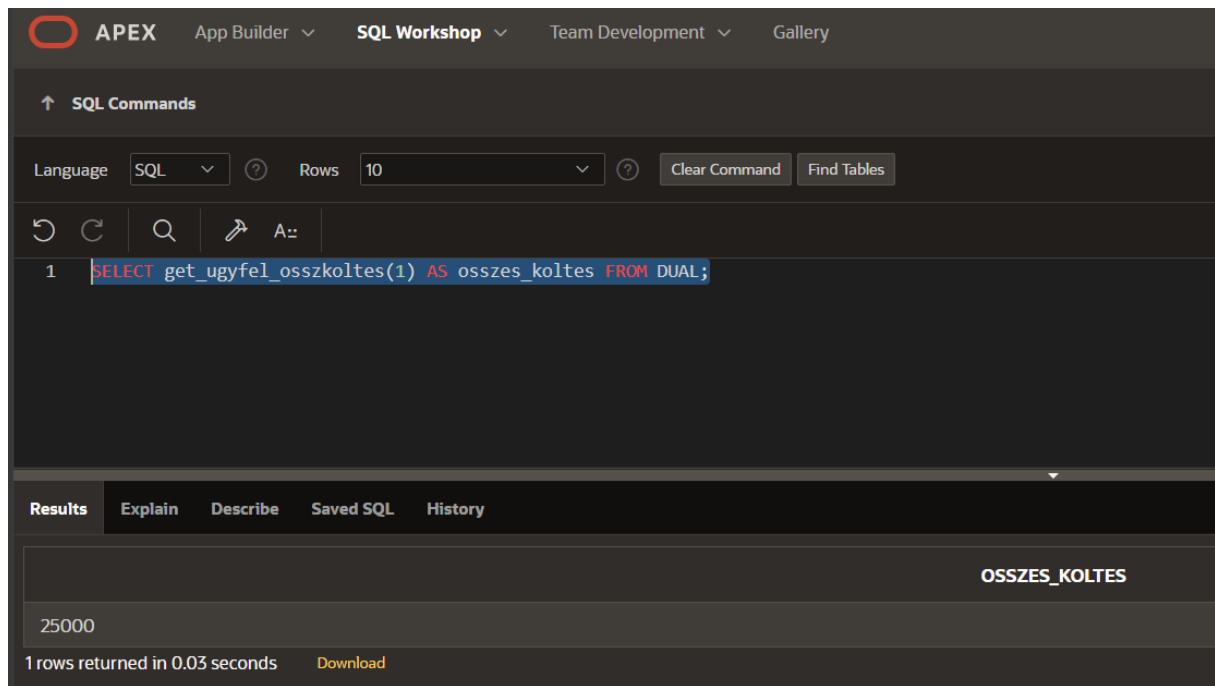
3. ábra: A `trg_ugyfel_naplo` trigger működése

Az implicit kurzort használó függvény és az aggregációt végző függvény is helyes eredményt ad vissza a tesztadatok alapján (Lásd: 4. ábra).



4. ábra: Függvények futtatása (implicit kurzor és aggregált adatok)

Az explicit kurzor a csomagon belül sikeresen iterál a rekordokon, és a `DBMS_OUTPUT` segítségével megjeleníti a foglалásokat (Lásd: 5. ábra).

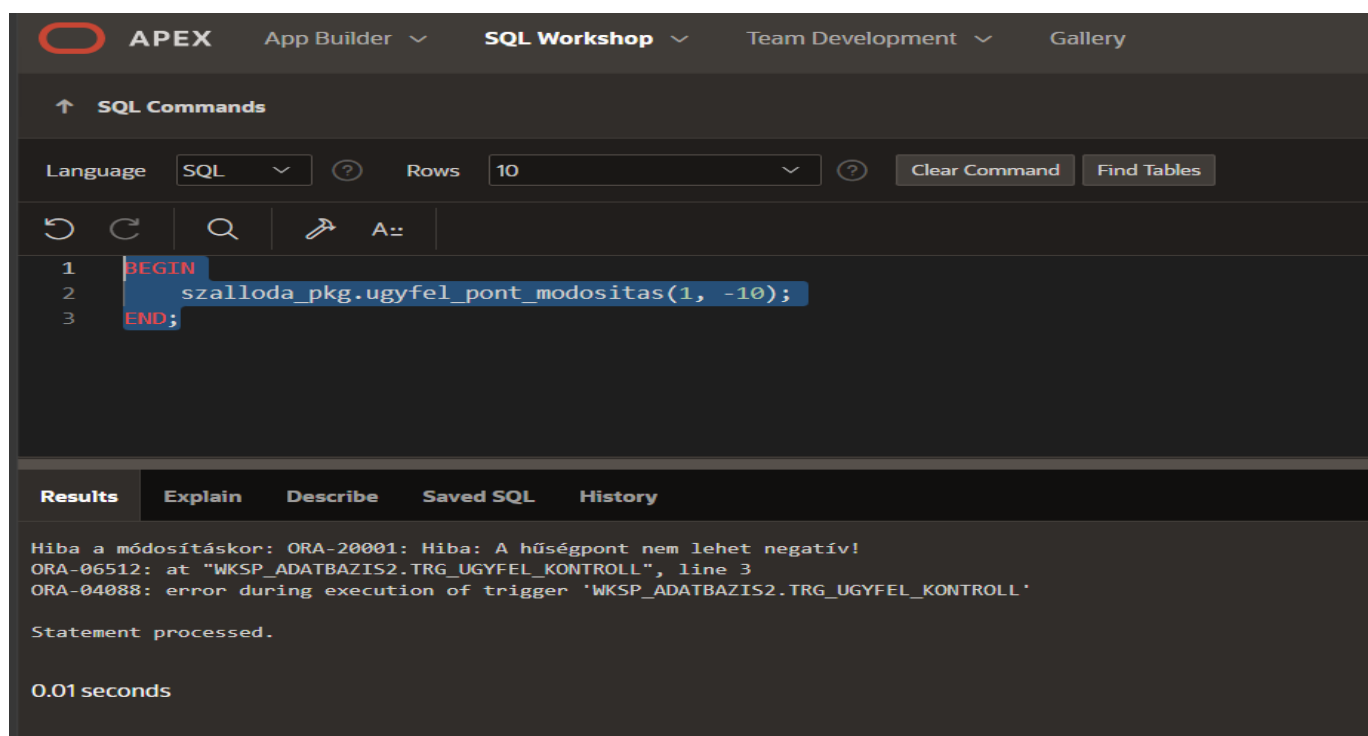


5. ábra: Explicit kurzor kimenete a konzolon

A PL/SQL program extra funkcióinak rövid bemutatása

A beadandó "extra funkcióként" egy robusztus, szerver oldali adatvédelmi vonalat épített ki. A legfőbb extra funkció az adatok logikai validálása `RAISE_APPLICATION_ERROR` beépítésével a triggerbe. Amikor az alkalmazásból valaki (akár szándékosan, akár programhibából) negatív értékre próbálja állítani egy lojális ügyfél hűségpontjait, a trigger azonnal beavatkozik.

Ahogy a 6. ábrán is látható, a kivételkezelés nem csupán megakadályozza a hibás adatbázis-állapotot, hanem beszédes hibaüzenettel tér vissza, amelyet a szerver azonnal visszadob a kliensnek.



6. ábra: A kontrolláló trigger beavatkozása hibás adatmódosítási kísérlet esetén

AI használata A beadandó elkészítése során nagy nyelvi modellt (AI) használtam a korábbi kliens-oldali Java programom SQL logikájának PL/SQL nyelvre történő szintaktikai átírására/javítására. A tervezési logika, az adatbázis séma megálmodása és az APEX környezetben történő tesztelés, hibajavítás önálló munka. Az AI használatának aránya a projekt egészét (kódolás és dokumentáció) tekintve megközelítőleg 35%.

